Binary Proof Search Trees as a Directed Container

Extracted from a machine-checked development

Abstract

Binary search trees form a directed container in which leaves are terminal nodes (dead ends or successes) and internal nodes are branching points. Shapes are tree skeletons; positions are paths from the root; the directed structure is path concatenation. We verify the five directed-container laws by structural recursion on paths. The induced comonad extracts the root label and duplicates by replacing each label with the full sub-search from that node, preserving dead ends at every depth. A coKleisli morphism is a proof strategy that may inspect the entire search tree. All stated results have been machine-checked.

Contents

1 Introduction **1**

2 Shapes and positions **2**

3 Subtree and path concatenation **2**

4 The directed-container laws **2**

5 The induced comonad **3**

6 Discussion **4**

1 Introduction

Proof search may be organised as a binary tree: at each branching point one chooses among (here, two) alternatives; leaves record terminal states—dead ends or successes. The distinction between dead end and success lives in the labelling of nodes, not in the shape. Navigation of such a tree is path concatenation: from a position one may restrict to the subtree there, or concatenate a further path.

Directed containers make this precise. A directed container is a container equipped with a root position, a subtree operation, and a path-concatenation (shift) operation, satisfying five equational laws. The associated polynomial functor becomes a comonad; coKleisli morphisms are strategies that consume a whole labelled search tree.

What this note claims. In ordinary mathematical language: binary tree shapes with path positions form a directed container. The five laws hold by structural recursion on paths. Dead ends—leaves, whose only position is the root—compose coherently at arbitrary depth: the search history is self-consistent no matter where one zooms in.

2 Shapes and positions

Definition 2.1 (Binary search tree shape)**.** A **binary search tree shape** is an element of the inductive type STree generated by

- leaf — a terminal node;
- $\mathsf{node}(\ell, r)$ — a branching point with left subtree ℓ and right subtree r.

The distinction between dead end and success is not recorded in the shape; it lives in any labelling of positions by a payload type X.

Definition 2.2 (Position)**.** A **position** (path) in a shape $t \in \mathrm{STree}$ is an element of the inductive family $\mathrm{SPos}(t)$ generated by

- here — the root of t;
- $\mathsf{left}(p)$ — descend into the left child of a node and continue along path p;
- $\mathsf{right}(p)$ — descend into the right child and continue along path p.

For $t = \mathsf{leaf}$, the only position is here. This is what makes a leaf a dead end for further search moves.

3 Subtree and path concatenation

Definition 3.1 (Subtree)**.** The **subtree** operation $\downarrow : (t : \mathrm{STree}) \to \mathrm{SPos}(t) \to \mathrm{STree}$ is defined by structural recursion on the path:

$$t \downarrow \mathsf{here} \;=\; t, \tag{1}$$
$$\mathsf{node}(\ell, r) \downarrow \mathsf{left}(p) \;=\; \ell \downarrow p, \tag{2}$$
$$\mathsf{node}(\ell, r) \downarrow \mathsf{right}(p) \;=\; r \downarrow p. \tag{3}$$

Definition 3.2 (Shift)**.** The **shift** (path concatenation)

$$\oplus \;:\; (t : \mathrm{STree}) \to (p : \mathrm{SPos}(t)) \to \mathrm{SPos}(t \downarrow p) \to \mathrm{SPos}(t) \tag{4}$$

embeds a position in a subtree back into the original tree:

$$\mathsf{here} \oplus q \;=\; q, \tag{5}$$
$$\mathsf{left}(p) \oplus q \;=\; \mathsf{left}(p \oplus q), \tag{6}$$
$$\mathsf{right}(p) \oplus q \;=\; \mathsf{right}(p \oplus q). \tag{7}$$

4 The directed-container laws

A directed container consists of a container $(\mathrm{Shape}, \mathrm{Pos})$ together with operations

$$o \;:\; \forall s.\, \mathrm{Pos}(s) \qquad \text{(root position)}, \tag{8}$$
$$\downarrow \;:\; \forall s.\, \mathrm{Pos}(s) \to \mathrm{Shape} \qquad \text{(subtree)}, \tag{9}$$
$$\oplus \;:\; \forall s.\, (p : \mathrm{Pos}(s)) \to \mathrm{Pos}(s \downarrow p) \to \mathrm{Pos}(s) \qquad \text{(shift)}, \tag{10}$$

satisfying five laws D1–D5. We take $\mathrm{Shape} = \mathrm{STree}$, $\mathrm{Pos} = \mathrm{SPos}$, $o(t) = \mathsf{here}$, and the operations of Definitions 3.1 and 3.2.

Proposition 4.1 (D1 and D2, definitional)**.** *For every shape t and every position q in t,*

$$t \downarrow \mathsf{here} \;=\; t, \qquad \mathsf{here} \oplus q \;=\; q. \tag{11}$$

Proof. By lean. □

Proposition 4.2 (D3: right identity)**.** *For every shape t and every position p in t,*

$$p \oplus \mathsf{here} \;=\; p. \tag{12}$$

That is, concatenating the empty path on the right is the identity.

Proof. By lean. □

Proposition 4.3 (D4: nesting of subtrees)**.** *For every shape t, position p in t, and position q in the subtree at p,*

$$t \downarrow (p \oplus q) \;=\; (t \downarrow p) \downarrow q. \tag{13}$$

Proof. By lean. □

Proposition 4.4 (D5: associativity of shift)**.** *For every shape t and compatible positions p, q, r,*

$$(p \oplus q) \oplus r' \;=\; p \oplus (q \oplus r), \tag{14}$$

where r' is r transported along the equality of fibres supplied by D4 (the position r lives over $(t \downarrow p) \downarrow q$, while the left-hand side requires it over $t \downarrow (p \oplus q)$).

Proof. By lean. □

Theorem 4.5 (Binary proof search as directed container)**.** *The data*

$$(\mathrm{STree},\ \mathrm{SPos},\ o = \mathsf{here},\ \downarrow,\ \oplus)$$

form a directed container: laws D1–D5 hold.

Proof. By lean. □

5 The induced comonad

Remark 5.1 (Extract and duplicate)**.** The directed container induces a comonad on the extension functor. For a labelling $v : \mathrm{SPos}(t) \to X$,

$$\mathrm{extract}\,\langle t, v\rangle \;=\; v(\mathsf{here}), \tag{15}$$

$$\mathrm{duplicate}\,\langle t, v\rangle \;=\; \Big\langle t,\ p \mapsto \langle t \downarrow p,\ q \mapsto v(p \oplus q)\rangle \Big\rangle. \tag{16}$$

Thus extract reads the label at the root, and duplicate replaces the label at every node by the full sub-search from that node—including all dead ends below it.

Remark 5.2 (Dead ends)**.** A dead end is a leaf: a position p at which $\mathrm{SPos}(t \downarrow p)$ is a singleton (only here). The comonad laws guarantee that dead ends compose coherently at arbitrary depth: the search history is self-consistent no matter where one zooms in.

Remark 5.3 (Proof strategies)**.** A coKleisli morphism $W(X) \to Y$ for the induced comonad W is a **proof strategy**: a function that may inspect the entire search tree (including dead ends) to decide what to produce.

6 Discussion

What has been established. Binary search tree shapes with path positions form a directed container. The five laws hold by structural recursion on paths; D1 and D2 hold definitionally as the base cases of that recursion. The induced comonad navigates the search space while preserving dead ends at every depth.

Scope. The development treats strictly binary branching. The generalisation to arbitrary finite arity (finitely branching rose trees) is a separate construction with the same directed-container pattern.

Machine checking. Every definition and theorem statement above corresponds to a declaration in a machine-checked formalisation. The phrase "by lean" indicates that the proof has been verified by a proof assistant.

Acknowledgements

The mathematical content follows the theory of directed containers of Ahman, Chapman, and Uustalu. The presentation here is a prose rendering of a formal development intended for human review of the theorem statements.

References

[1] D. Ahman, J. Chapman, and T. Uustalu. When is a container a comonad? In *FoSSaCS*, volume 7213 of *LNCS*, pages 74–88. Springer, 2012.

[2] M. Abbott, T. Altenkirch, and N. Ghani. Containers: constructing strictly positive types. *Theoretical Computer Science*, 342(1):3–27, 2005.